

Class 11: Arrays 1 : One Dimensional

Q1. Given array of N integers, where each integer represent the stock price change on that day.
 +ve value means price goes up (profit)
 -ve " " price " down (loss)

Total profit/loss is the total price change b/w the duration you hold the stock. Find the max profit?

$$A[] = [-20, 30, 40, -10, 50, -100, 70]$$

Short Problem Statement :- Given an array, find maximum subarray sum.

$$A[] = [4, 5, 2, 1, 6] \quad \text{Maximum Subarray Sum : ans} = 18$$

$$A[] = [-4, -3, -6, -9, -2] \quad \text{" " " " : ans} = -2$$

Brute Force

Iterate over all subarrays, calculate sum & take max.

$$\# \text{ of subarrays} = N(N+1)/2$$

$$\text{ans} = A[0]$$

for ($i=0$ to $N-1$) {

for ($j=i$ to $N-1$) {

$$\text{sum} = 0$$

for ($k=i$ to j)

$$\text{sum} += A[k]$$

// subarray (i, j)

$$TC: O(N^3)$$

$$SC: O(1)$$

$$\text{ans} = \max(\text{ans}, \text{sum})$$

}

}

$$\rightarrow \text{Prefix sum: } Pf(R) - Pf(L-1) \quad O(N^2) \text{ } \& SC: O(1)$$

Carry forward method

```

ans = a[0]
for (i = 0 to n-1) {
    sum = 0
    for (j = i to n-1) {
        sum += a[j]
        ans = Math.max(ans, sum)
    }
}
return ans.
    
```

TC: $O(N^2)$
SC: $O(1)$

Observations

① If all elements are +ve,
 $a[i] \geq 0 \Rightarrow \text{ans} = \sum_{i=0}^{n-1} a[i]$
 take sum of all elements

② $\forall i$ (If for all i), $a[i] \leq 0 \Rightarrow \text{ans} = \max(a[i])$
 If all elements are -ve. pick the max element

$A = [-4 -3 -6 -9 -2]$ $\text{ans} = -2$

③ $A = [-ve \ -ve \ +ve \ +ve \ +ve \ -ve \ -ve]$

Max subarray sum = sum of consecutive +ve,

$A = [+ve \ +ve \ +ve] \ A[-ve \ -ve]$

④ $A = [+ve \ +ve \ -ve \ -ve] \ +ve \ +ve \ -ve$

⑤ $A = [-2 \ \boxed{3 \ 4 \ -1 \ 5} \ -1 \ -7 \ 2 \ -5 \ 12]$

sum = ~~-2~~ 3 7 6 11 1 ~~-6~~ 0 2 ~~-3~~ 0 12

ans = ~~-2~~ 3 7 11 12

3 is greater than -2

whenever our sum becomes -ve, ignore -ve

no need to take -ve in our subarray.

Kadane's Algorithm

```
ans = a[0]
sum = 0
for (i = 0 to N-1) {
    sum += a[i]
    if (sum > ans) {
        ans = sum
    }
}
```

TC: $O(N)$

SC: $O(1)$

✓
Kadane's Algorithm

```
if (sum < 0) {
    sum = 0
}
return ans;
```

Now how to get subarray indexes,

```
ans = a[0]
sum = 0, start = 0
for (i = 0 to N-1) {
    sum += a[i]
    if (sum > ans) {
        ans = sum
        L = start, R = i
    }
    if (sum < 0) {
        sum = 0, start = i + 1
    }
}
return L, R;
```


Q2 Given an array where all elements are zero, i.e., $a[i] = 0$.
 Return final array after performing multiple queries.
 Query = $(i, x) \rightarrow$ add x to all elements from index i to $n-1$.

$$A = [0^0 \ 0^1 \ 0^2 \ 0^3 \ 0^4 \ 0^5 \ 0^6]$$

Queries = $[1, 3], [4, 2], [3, -1]$
 $\hookrightarrow$ add 3 from index 1 till the end.

$$A = [0^0 \ 0^1 \ 0^2 \ 0^3 \ 0^4 \ 0^5 \ 0^6]$$

+3 +3 +3 +3 +3 +3

$$+2 \ +2 \ +2$$

$$-1 \ -1 \ -1 \ -1$$

$$A = [0 \ 3 \ 3 \ 2 \ 4 \ 4 \ 4]$$

Brute force

$$TC: O(QN) \quad SC: O(1)$$

$\hookrightarrow$ for every query we will take N times.
 for each query add element x to the last.

$$\text{Prefix Sum: } Pf[i] = Pf[i-1] + a[i]$$

$$a[i] = a[i-1] + a[i]$$

when we create Pf array, if we add +3 at index 1, Pf sum array will have +3 in all elements till the end.

$$A = [0^0 \ 0^1 \ 0^2 \ 0^3 \ 0^4 \ 0^5 \ 0^6]$$

3 -1 2

$$Pf = [0 \ 3 \ 3 \ 2 \ 4 \ 4 \ 4]$$

Code

for (i=0 to Q.length) {
 $a[Q[i][0]] += Q[i][1]$ // updating only elements at their index.
 }
 TC: $O(Q+N)$
 SC: $O(1)$

used when we have to update elements on the same index

✓ for (i=1 to N-1) {
 $a[i] = a[i-1] + a[i]$ // updating original array
 }
 return a

creating prefix sum type

Dry run

$A = [0, 0, 0, 0, 0]$ Queries $(1, 3) (4, 2) (3, -1)$

$i=0$

$a[Q[0][0]] += Q[0][1]$ // we have done (+) here
 $a[1] += 3$ coz if there is already an
 $a[1] = a[1] + 3 = 0 + 3 = 3$ query/element at index & suppose
 $i=1$ $(1, 2)$ then we should add

$a[Q[1][0]] += Q[1][1]$ that is not override,

$a[4] += 2$

$a[4] = a[4] + 2 = 0 + 2 = 2$

$i=2$

$a[Q[2][0]] += Q[2][1]$

$a[3] += -1$

$a[3] = a[3] - 1 = 0 - 1 = -1$

Part 2
 Q3 Given an array where all elements is zero, $\forall i, a[i] = 0$. Return final array after performing multiple queries.
 Query: $(i, j, x) \rightarrow$ add x to all elements from index i to j

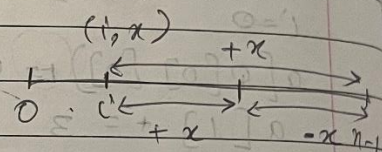
$$A = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ +3 & +3 & +3 & +3 & & & \end{bmatrix}$$

Queries
 $(1, 4, 3)$
 $(4, 5, 2)$
 $(3, 6, -1)$
 $0 \ 1 \ 2$

$$A = \begin{bmatrix} 0 & 3 & 3 & 2 & 4 & 1 & -1 \end{bmatrix}$$

$$(i, j, x) \Rightarrow [i, j] + x$$

$$[i, n) \Rightarrow [i, n-1] + x$$



$$[j+1, n] \Rightarrow [j+1, n-1] - x$$

Code for $(i=0 \text{ to } Q.length)$ {
 $a[Q[i][0]] += Q[i][2]$ // add element
 if $(Q[i][1] < n-1)$ {
 $a[Q[i][1]+1] -= Q[i][2]$ // subtract element
 }
 }
 for $(i=1 \text{ to } N-1)$ {
 $a[i] = a[i-1] + a[i]$
 }
 return a

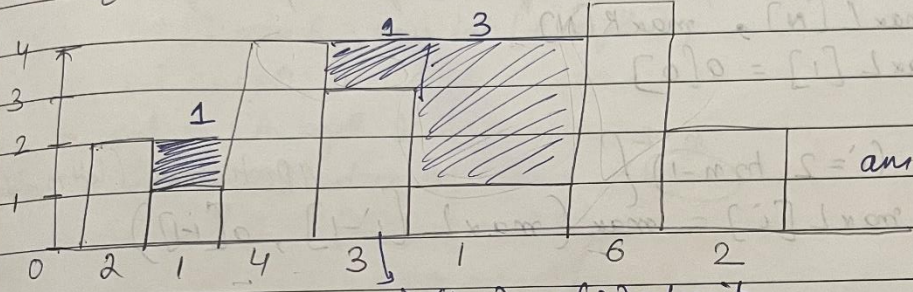
$(i, j, x): Q[i][0], Q[i][1], Q[i][2]$

$$A = \begin{bmatrix} \emptyset & \emptyset & \emptyset & \emptyset & \emptyset & \emptyset & \emptyset & \emptyset \\ -1 & 3 & 4 & -4 & 3 & -3 & 1 & -3 \end{bmatrix} \begin{matrix} (1, 4, 3) \\ (0, 5, -1) \\ (2, 2, 4) \\ (, ,) \end{matrix}$$

$$A = \begin{bmatrix} -1 & 2 & 6 & 2 & 5 & 2 & 3 & 0 \end{bmatrix}$$

Q3 Given N buildings & height of each building. Find the rain water trapped b/w buildings.

$$A = \begin{bmatrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 \\ 2 & 1 & 4 & 3 & 1 & 6 & 2 \end{bmatrix}$$



$$ans = 1 + 1 + 3 = 5$$

$$\min(4, 6) - a[3] = 1 \text{ unit}$$

Brute force For each i (i), find max height in left & max height in right and then take the $\min(\text{left}, \text{right}) - a[i]$ (my current height).

$$\min(4, 6) - a[4] = 4 - 1 = 3 \text{ units}$$

ans = 0 $\rightarrow$ coz there is no water trapping at the ends

for ($i=1$ to $N-2$) {
 $\text{maxleft} = \max(0, i-1)$ // loop $O(N)$
 $\text{maxright} = \max(i+1, n-1)$ // loop $O(N)$

$$\text{water} += \min(\text{maxleft}, \text{maxright}) - a[i] \quad \text{TC: } O(N^2)$$

if ~~else~~ (water > 0) {
 $\text{ans} += \text{water}$ if water is non-negative
 $\text{SC: } O(1)$

	0	1	2	3	4	5	6
A =	2	1	4	3	1	6	2
maxleft	2	2	4	4	4		
maxright		6	6	6	6	2	
min(maxL, maxR)	2	2	4	4	2		
water	1	-2	1	3	-4		

→ $(-a[i])$

$$ans = 1 + 1 + 3 = 5$$

Optimize

Create maxleft & maxright array using carry forward,

maxL[N], maxR[N]

$$maxL[1] = a[0]$$

✓
 $O(N)$ for $(i=2 \text{ to } n-1)$
 $maxL[i] = \max(maxL[i-1], a[i])$ // prefix max (left to right)

$$maxR[n-2] = a[n-1]$$

$O(N)$ for $(i=n-3 \text{ to } 0)$
 $maxR[i] = \max(maxR[i+1], a[i])$ // suffix max (right to left)

$$ans = 0$$

$O(N)$ for $(i=1 \text{ to } n-2)$
 $water = \min(maxL[i], maxR[i]) - a[i]$
 if $(water > 0)$
 $ans += water$

Total
 TC: $O(N)$ return ans

SC: $O(N)$

Assignment

Rain Water Trapped

Imagine a histogram where the bar's heights are given by the array A . Each bar is of uniform width, which is 1 unit. When it rains, water will accumulate in the valleys b/w the bars. Your task is to calculate the total amt. of water that can be trapped in these valleys.

I/p format: $A = [0, 1, 0, 2]$

O/p format: 1.

public class Solution {

public int trap (final int[] A) {

int[] maxL = new int[A.length];

int[] maxR = new int[A.length];

int ans = 0;

int n = A.length;

if (n < 3) return 0;

maxL[1] = A[0];

for (int i = 2; i < n; i++) {
maxL[i] = Math.max(maxL[i-1], A[i-1]);

}

maxR[n-2] = A[n-1];

for (int i = n-3; i >= 0; i--) {
maxR[i] = Math.max(maxR[i+1], A[i+1]);

}

int water = 0;

for (int i = 1; i < n-1; i++) {
water = Math.min(maxL[i], maxR[i]) - A[i];
if (water > 0) ans += water;

}

return ans;

}

calculating
max left

calculating
max right

calculating
water

adding all
water to get
final ans

}

Problem Statement -1 -Max Sum Contiguous Subarray

Problem Description

Find the maximum sum of **contiguous non-empty subarray** within an array **A** of length **N**.

Problem Constraints

$1 \leq N \leq 1e6$

$-1000 \leq A[i] \leq 1000$

Input Format

The first and the only argument contains an integer array, A.

Output Format

Return an integer representing the maximum possible sum of the contiguous subarray.

Example Input

Input 1: A = [1, 2, 3, 4, -10]

Input 2: A = [-2, 1, -3, 4, -1, 2, 1, -5, 4]

Example Output

Output 1: 10

Output 2: 6

Example Explanation

Explanation 1: The subarray [1, 2, 3, 4] has the maximum possible sum of 10.

Explanation 2: The subarray [4,-1,2,1] has the maximum possible sum of 6.

Actual Code

```
public class Solution {
    public int maxSubArray(final int[] A) {
        int n = A.length;
        //if(n==0) return 0;
        //if(n==1) return A[0];
        int sum = 0;
        int ans = A[0];
        for(int i = 0; i<n; i++){
            sum += A[i];
            if(sum > ans) ans = sum;
            if (sum < 0) sum = 0;
        }
        return ans;
    }
}
```


Problem Statement -2 - Continuous Sum Query

Problem Description

There are **A** beggars sitting in a row outside a temple. Each beggar initially has an empty pot. When the devotees come to the temple, they donate some amount of coins to these beggars. Each devotee gives a fixed amount of coin(according to their faith and ability) to some **K** beggars sitting next to each other.

Given the amount **P** donated by each devotee to the beggars ranging from **L** to **R** index, where $1 \leq L \leq R \leq A$, find out the final amount of money in each beggar's pot at the end of the day, provided they don't fill their pots by any other means.

For *i*th devotee $B[i][0] = L$, $B[i][1] = R$, $B[i][2] = P$, given by the 2D array **B**

Problem Constraints

$$1 \leq A \leq 2 * 10^5$$

$$1 \leq L \leq R \leq A$$

$$1 \leq P \leq 10^3$$

$$0 \leq \text{len}(B) \leq 10^5$$

Input Format

The first argument is a single integer A.

The second argument is a 2D integer array B.

Output Format

Return an array(0 based indexing) that stores the total number of coins in each beggars pot.

Example Input

Input 1:-

$$A = 5$$

$$B = [[1, 2, 10], [2, 3, 20], [2, 5, 25]]$$

Example Output

Output 1:-

$$10 \ 55 \ 45 \ 25 \ 25$$

Example Explanation

Explanation 1:-

First devotee donated 10 coins to beggars ranging from 1 to 2. Final amount in each beggars pot after first devotee: [10, 10, 0, 0, 0]

Second devotee donated 20 coins to beggars ranging from 2 to 3. Final amount in each beggars pot after second devotee: [10, 30, 20, 0, 0]

Third devotee donated 25 coins to beggars ranging from 2 to 5. Final amount in each beggars pot after third devotee: [10, 55, 45, 25, 25]

Actual Code


```

public class Solution {
    public int[] solve(int A, int[][] B) {
        int n = B.length;
        int [] arr = new int[A];
        for(int i = 0; i<n; i++){
            arr[B[i][0]-1] += B[i][2];
            if(B[i][1]-1 < A-1){
                arr[B[i][1]] -= B[i][2];
            }
        }
        for(int i = 1; i<A; i++){
            arr[i] += arr[i-1];
        }
        return arr;
    }
}

```

Problem Statement – 3 - Rain Water Trapped

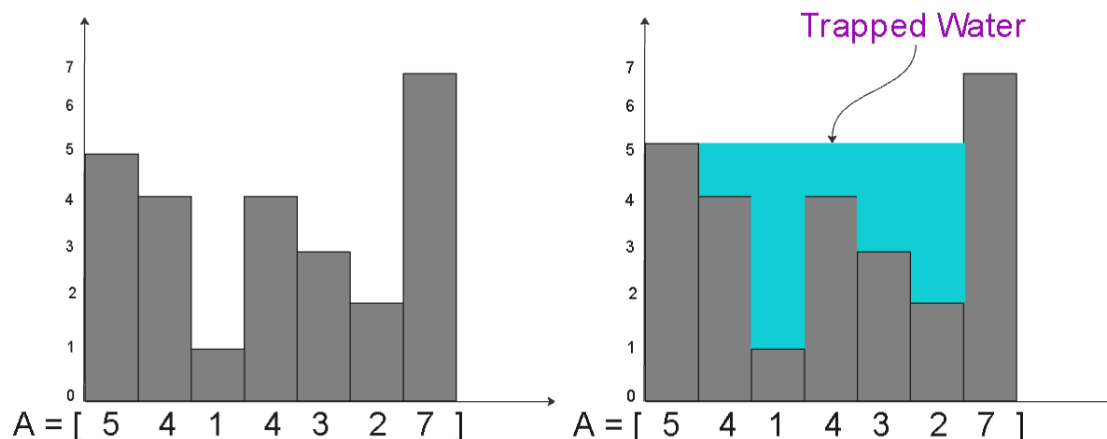
Problem Description

Imagine a histogram where the bars' heights are given by the array **A**. Each bar is of **uniform width**, which is **1 unit**. When it rains, water will accumulate in the valleys between the bars.

Your task is to calculate the **total** amount of water that can be **trapped** in these valleys.

Example:

The Array $A = [5, 4, 1, 4, 3, 2, 7]$ is visualized as below. The total amount of rain water trapped in A is **11**.



Problem Constraints

$1 \leq |A| \leq 10^5$

$0 \leq A[i] \leq 10^5$

Input Format

First and only argument is the Integer Array, **A**.

Output Format

Return an **Integer**, denoting the total amount of water that can be trapped in these valleys

Example Input

Input 1:

A = [0, 1, 0, 2]

Input 2:

A = [1, 2]

Example Output

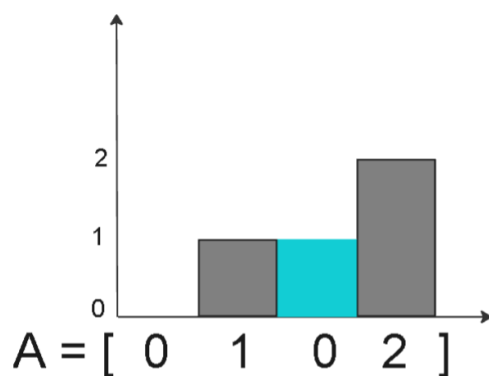
Output 1: 1

Output 2: 0

Example Explanation

Explanation 1:

1 unit is trapped on top of the 3rd element.



Explanation 2:

No water is trapped.

Actual Code

```
public class Solution {
    public int trap(final int[] A) {

        int []maxL = new int [A.length];
        int [] maxR = new int[A.length];
        int ans = 0;
        int n = A.length;
        if (n < 3) return 0;
        maxL[1] = A[0];
        for(int i = 2; i<n; i++){
            maxL[i] = Math.max(maxL[i-1], A[i-1]);
```



```

    }
    maxR[n-2] = A[n-1];
    for(int i = n-3; i>=0; i--){
        maxR[i] = Math.max(maxR[i+1], A[i+1]);
    }
    int water = 0;
    for(int i = 1; i<n-1; i++){
        water = Math.min(maxL[i], maxR[i]) - A[i];
        if(water > 0) ans+=water;
    }
    return ans;
}
}

```

Problem Statement – 4 - Add One To Number

Problem Description

Given a **non-negative** number represented as an array of digits, add **1** to the number (increment the number represented by the digits).

The digits are stored such that the most significant digit is at the head of the list.

NOTE: Certain things are intentionally left unclear in this question which you should practice asking the interviewer. For example: for this problem, the following are some good questions to ask :

- **Q:** Can the input have **0's** before the most significant digit. Or, in other words, is **0 1 2 3** a valid input?
- **A:** For the purpose of this question, **YES**
- **Q:** Can the output have **0's** before the most significant digit? Or, in other words, is **0 1 2 4** a valid output?
- **A:** For the purpose of this question, **NO**. Even if the input has zeroes before the most significant digit.

Problem Constraints

1 <= size of the array <= 1000000

Input Format

First argument is an array of digits.

Output Format

Return the array of digits after adding one.

Example Input

Input 1:

[1, 2, 3]

Example Output

Output 1:

[1, 2, 4]

Example Explanation

Explanation 1:

Given vector is [1, 2, 3].

The returned vector should be [1, 2, 4] as $123 + 1 = 124$.

Actual Code

```
public class Solution {
    public int[] plusOne(int[] arr) {
        int start = 0;
        while(start < arr.length && arr[start] == 0){    //case when 0 is at the beginning eg 0124
            start++;
        }
        int[] A = new int [arr.length-start];
        int n = A.length;
        for(int i = 0; i<n; i++){
            A[i] = arr[start+i];
        }
        for(int i = n-1; i>=0; i--){
            if(A[i] == 9) {
                A[i] = 0;
            }
            else if(A[i] < 9){
                A[i] = A[i] + 1;
                return A;
            }
        }
        int [] array = new int [n+1];
        array[0] = 1;
        return array;
    }
}
```

Problem Statement – 5 – Flip

Problem Description

You are given a binary string **A**(i.e., with characters **0** and **1**) consisting of characters **A₁, A₂, ..., A_N**. In a single operation, you can choose two indices, **L** and **R**, such that **1 ≤ L ≤ R ≤ N** and flip the characters **A_L, A_{L+1}, ..., A_R**. By flipping, we mean changing character **0** to **1** and vice-versa.

Your aim is to perform **ATMOST** one operation such that in the final string number of **1s** is maximized.

If you don't want to perform the operation, return an **empty** array. Else, return an array consisting of two elements denoting **L** and **R**. If there are multiple solutions, return the **lexicographically smallest** pair of **L** and **R**.

NOTE: Pair (**a**, **b**) is lexicographically smaller than pair (**c**, **d**) if **a < c** or, if **a == c** and **b < d**.

Problem Constraints

1 ≤ size of string ≤ 100000

Input Format

First and only argument is a string **A**.

Output Format

Return an array of integers denoting the answer.

Example Input

Input 1: **A** = "010"

Input 2: **A** = "111"

Example Output

Output 1: [1, 1]

Output 2: []

Example Explanation

Explanation 1:

A = "010"

Pair of [L, R] | Final string

[1 1]	"110"
[1 2]	"100"
[1 3]	"101"
[2 2]	"000"
[2 3]	"001"

We see that two pairs [1, 1] and [1, 3] give same number of 1s in final string. So, we return [1, 1].

Explanation 2:

No operation can give us more than three 1s in final string. So, we return empty array [].

Actual Code


```

public class Solution {
    public int[] flip(String A) {
        char arr[] = A.toCharArray();
        int start = 0, count = 0, counta = 0;
        int maxcount = Integer.MIN_VALUE, sum = 0;
        int left = -1, right = -1;

        int [] array = new int[2];
        int [] B = new int [0];

        for(int i = 0; i<arr.length; i++){
            if(arr[i] == '1') counta++;
        }
        if (counta == arr.length) return B;

        for(int i = 0; i<arr.length; i++){
            if(arr[i] == '0'){
                count++;
            }else{ count--;}

            if (count > maxcount){
                maxcount = count;
                left = start;
                right = i;
            }
            if( count < 0) {
                count = 0;
                start = i+1;
            }
        }
        if (left == -1 || right == -1) return B;
        array[0] = left+1;
        array[1] = right +1;
        return array;
    }
}

```